Dockerization Step-by-Step Guide

This reference document outlines the exact progression for building and running your Python 3 application inside a Docker container.

1. The Docker Core Concept Flow

Component	Description
Dockerfile	A text blueprint containing explicit instructions to build an image layer by layer.
docker build	The execution command that processes the Dockerfile and compiles the system environment.
Image	The final, static, immutable package ready to deploy and run consistently anywhere.

2. Python 3 Dockerfile Configuration

Create a file named exactly **Dockerfile** (no file extension) in your root directory ~/devops-day20-docker and add the following contents:

```
# 1. BASE IMAGE
FROM python:3.11-slim

# 2. WORKING DIRECTORY
WORKDIR /app

# 3. DEPENDENCY INSTALLATION
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 4. APPLICATION FILES
COPY . .

# 5. APPLICATION COMMAND
CMD ["python", "src/main.py"]
```

3. Detailed Step-by-Step Implementation

Step 3.1: Create the Dockerfile
Open your terminal inside ~/devops-day20-docker and launch the text editor:
nano Dockerfile
Paste the configuration block from Section 2 into the editor. Press Ctrl+O, then Enter to save, and Ctrl+X to exit.

Step 3.2: Verify Main Script Name
Check the contents of your source folder to ensure your entrypoint script matches line 5 of the Dockerfile:
ls src/
Note: If your file is named 'app.py' instead of 'main.py', update the final line of your Dockerfile to: CMD ["python", "src/app.py"]

Step 3.3: Execute Image Creation
Run the build command from your project root. Do not forget the trailing dot (.) which specifies the current context:
sudo docker build -t python-app:v1 .

Step 3.4: Verify the Image
Confirm your image compiled successfully and exists in local storage:
sudo docker images